

【环境】 昆明湖vcs+verdi编译仿真流程

环境准备

1 在 `open01` 服务器（`open0x`服务器均可），clone一个稳定版本的香山代码仓库

例如：

<https://github.com/OpenXiangShan/XiangShan/commit/b5a614b988d57ea277a71eb428aa98889c483c14>

可以从Xiangshan的master下载最新的仓库之后checkout

```
1 proxychains4 git clone https://github.com/OpenXiangShan/XiangShan.git
2 git checkout b5a614b9      #从上面的稳定版本复制提交序列号
3 proxychains make init      #拉取submodule。出了问题的可以删除该模块，再次make init，重新下载
```

2 环境变量设置

使用下面的路径中的脚本配置环境变量，在 `home/konglinghui/share` 下

```
1 source /nfs/home/konglinghui/share/xs-env/env.sh
```

为了方便，可以修改 `~/.bashrc`，把上面的命令设置一个别名，方便每次配置。

```
1 alias sourceXS = "source /nfs/home/konglinghui/share/xs-env/env.sh"
```

如果自己本地直接clone了 `xs-env` 仓库：可以使用自己路径下的环境变量配置，不必去 `home/konglinghui/share` 路径下：

```
1 cd xs-env
2 # 如果是首次clone配置环境，需要先执行以下命令：
```

```
3 source setup.sh
4 #配置完之后，每次只需要执行下面的命令加载环境变量即可：
5 source env.sh
```

但是 `xs-env` 目前无人维护，如果选择使用 `xs-env`，需要手动将 submodule checkout 到需要的分支。

```
1 source update-submodule.sh
```

3 VCS Verdi环境准备

在 `~/.bashrc` 中增加以下环境变量

```
1 # vcs
2 export LD_LIBRARY_PATH=/home/tools/synopsys/verdi/S-2021.09-
  SP1/share/PLI/lib/LINUX64
3 export SYNOPSYS_LC_R00T=/home/tools/synopsys/lc/R-2020.09-SP1/bin
4
5 # verdi
6 export VERDI_HOME=/nfs/tools/synopsys/verdi/R-2020.12-SP1
```

在 `eda01` 服务器，需要先 `module load` 必要的 `eda license`：

```
1 module avail # 查看可以使用的eda工具
2 module load ** # 选择vcs, verdi和license, 之后就可以使用vcs和verdi进行编译/运行/看
  波形了
3
4 # e.g.
5 module load synopsys/vcs/Q-2020.03-SP2 synopsys/verdi/R-2020.12-SP1 license
6 # 同样可以在.bashrc中设置别名：
7 alias modulevcs="module load synopsys/vcs/Q-2020.03-SP2 synopsys/verdi/R-
  2020.12-SP1 license"
```

4 NEMU编译（可选）

NEMU是difftest机制中用于参考的golden模型。

如果采用的是上面的库，一般在ready-to-run文件夹下有编译好的nemu动态链接库，可以直接用。本步骤跳过。

另一种方式是，如果clone `xs-env` 仓库，想使用 `xs-env` 下的 `NEMU`，需要自己重新编译：

1. 更新gcc：

编译需要用到gcc，更新为最新版本：（要在自己的home下进行）

```
1 scl enable devtoolset-ll /bin/bash    #将工具设置为最新版本
2 gcc --version
```

2. 编译NEMU

按照这个教程编译新NEMU临时使用指南 · OpenXiangShan/NEMU Wiki (github.com)

```
1 # 需要先 export NEMU_HOME=NEMU_POSITION
2 cd $NEMU_HOME
3 make clean
4 make riscv64-xs-ref_defconfig
5 make -jN
```

这里的 `riscv64-xs-ref_defconfig` 是在 `configs` 下预先写好的配置文件. 其中 `riscv64-xs-ref_defconfig` 是为 difftest 生成动态链接库的配置. `riscv64-xs_defconfig` 是与香山外设对齐, 可直接执行的 NEMU 模拟器.

编译流程

编译分两个阶段：

1. 在普通服务器上（推荐5950x），编译成verilog（即open01服务器）
2. 在eda服务器上，vcs 编译生成 simv 仿真文件

备注：原因是eda服务器上不了网，也没有verilator（用不上，但是make文件写法上要求有）

注意：生成行为仿真用 verilog 代码和可综合 verilog 代码的命令是不同的。通过 make verilog 命令可以生成可综合的 verilog 代码；而 make emu 用于行为仿真。（参见：[issue #933](#)）

- 仿真
 - 行为仿真 make emu

- 可综合的代码的功能仿真
 - `make verilog -jN`——没有difftest，用于流片
 - `make sim-verilog -jN` —— 有difftest，用于仿真

1 在 `open01` 服务器上运行 `make sim-verilog -jN` 生成verilog代码

- 编译香山的 Chisel 代码，生成 Verilog，输出的文件在 `XiangShan/build/XSTop.v`
- `make verilog` 命令生成的Verilog文件用于生成FPGA的bitstream和流片，去除了Difftest等仿真用的调试模块。`make sim-verilog` 命令生成带有Difftest的用于仿真的Verilog文件。

```
1 make sim-verilog -jN    //带difftest
2 make verilog -jN       //N可以用12
3 # 也可以直接 make simv, simv是依赖于sim-verilog的
4 make simv -jN
```

2 在 `eda01` 服务器运行香山编译命令：

vcs编译准备——脚本的修改：增加-kdb编译选项

后续因为需要使用verdi关联代码和波形，之前的vcs编译脚本缺少了 `-kdb` 选项，导致vcs编译生成的fsdb缺少了与代码关联的调试信息。因此需要去修改vcs编译的脚本：

目录：`XiangShan/difftest/vcs.mk`

```
1 #找到如下一行：
2 EXTRA = +define+CONSIDER_FSDB
3 #在其后面增加 -kdb
4 EXTRA = +define+CONSIDER_FSDB -kdb
```

这样，当编译时带有 `CONSIDER_FSDB=1` 选项，就会包含kdb，将会关联波形和代码。

vcs编译生成仿真程序

编译生成 `simv` 仿真器执行文件

```
1 make simv RELEASE=1 CONSIDER_FSDB=1 -j16
2 #make simv RELEASE=1 CONSIDER_FSDB=1 ENABLE_XPROP=1 -j16
```

修改仿真VCS仿真频率：以2GHz仿真为例

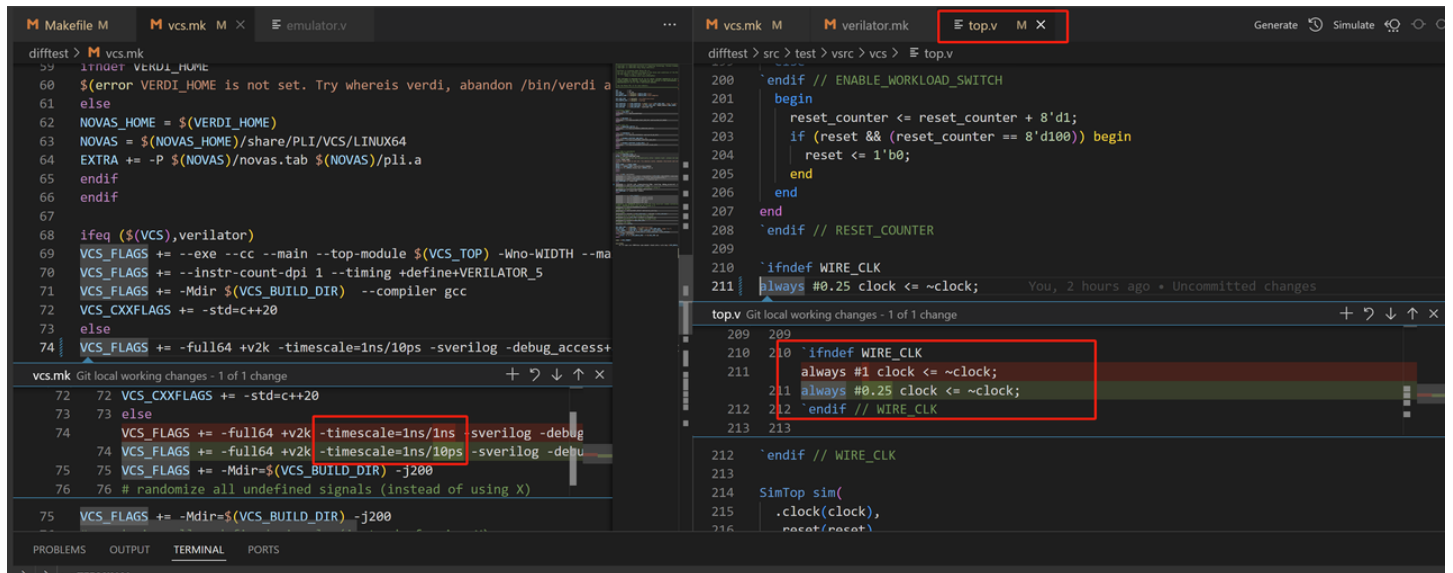
在vcs.mk中，vcs仿真参数 -timescale=1ns/10ps用于控制仿真时间单位以及仿真精度

精度只能选取100、10、1等值，例如：100ps、10ps、10ns、1ns

```
1 -timescale=1ns/10ps
2 仿真时间单位 1ns/仿真精度10ps
```

verilog仿真频率由clock翻转速率决定，在difftest/src/test/vsrc/vcs/top.v 文件中设置了clock反转速率

```
1 `ifndef WIRE_CLK
2 always #0.25 clock <= ~clock;
3 `endif // WIRE_CLK
4
5 每0.25个仿真时间单位翻转一次clock，翻转两次为1 cycle  $0.25 \times 2 \times 1\text{ns} = 0.5\text{ns}$ 
6 仿真频率  $f = 1/0.5 \text{ ns} = 2 \text{ GHz}$ 
```



3 运行仿真程序与difftest测试，生成fsdb波形

执行文件为 build/simv，读入workload，使用NEMU作为参照进行difftest测试，并生成fsdb波形文件

```
1 ./build/simv +workload=./ready-to-run/microbench.bin
```

```
+diff=/nfs/home/luguangyang/Xiangshan-sim/XiangShan/ready-to-run/riscv64-nemu-
interpreter-dual-so +dump-wave=fsdb
2 # 或者用本当前目录下的./ready-to-run/riscv64-nemu-interpreter-so
3 ./build/simv +workload=./ready-to-run/microbench.bin +diff=./ready-to-
run/riscv64-nemu-interpreter-so +dump-wave=fsdb
4
```

- +workload：从ready-to-run中选取要使用的workload（.bin文件）。
- +diff：选取nemu动态链接库，一般在ready-to-run中会有。如果没有或想用xs-env下的环境变量，参考上面的NEMU编译。
- +dump-wave=fsdb：如果不打波形可以去掉

microbench.bin中有多个测试程序，出现一个Pass之后就可以暂停了，全部运行完耗时很长。

4 verdi 查看波形：

```
1 仅打开波形图： Verdi -ssf simv.fsdb
2 关联仿真波形和代码： Verdi -ssf simv.fsdb -dbdir ./build/simv.daidir
```

verdi 快捷键：

```
1 shift + S           // 用 *name* 直接跳转module
2 Ctrl + W            // 选中信号之后直接添加到下面
3 【src页面按】X      //在信号下面显示值，便于观察
```

可能出现的工具链问题

```
[warn] There were 132 warning(s) during hardware elaboration.
Exception in thread "main" circst.stage.phases.Exceptions$FirtoolNonZeroExitCode: /nfs/home/luguangyang/.cache/llvm-firtool/1.62.0/bin/firtool returned a non-zero exit code. Note that this version of Chisel (6.5.0) was published against firtool v
ersion 1.62.0.
-----
ExitCode:
1
STDOUT:
STDERR:
/nfs/home/luguangyang/.cache/llvm-firtool/1.62.0/bin/firtool: /lib64/libc.so.6: version 'GLIBC_2.18' not found (required by /nfs/home/luguangyang/.cache/llvm-firtool/1.62.0/bin/firtool)
-----
1 targets failed
xiangshan.test.runMain Subprocess failed
make: *** [build/rtl/SuMTop.sv] Error 1
```

核心问题是：

```
/lib64/libc.so.6: version `GLIBC_2.18' not found (
```

NEMU的版本太低：

首先需要高版本的GCC重新编译NEMU：

```
1 //检查gcc版本
2 gcc --version
```

```
-> % gcc --version
gcc (GCC) 4.8.5 20150623 (Red Hat 4.8.5-39)
Copyright (C) 2015 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

发现版本很低

所以需要换成高版本的：（不同的shell有不同的命令）

如果用的是bash：

```
1 scl enable devtoolset-11 /bin/bash
2 //如果这个出错，可以试一下
3 scl scl_source enable gcc-toolset-11
```

如果用的是zsh：

```
1 source /opt/rh/devtoolset-11/enable
```